운영체제 project 1

Lecture: 12912

2022090000 정인호

1. design

위 프로젝트는 xv6 운영체제 환경에서 round robin 스케줄링 방식을 cfs 스케줄링으로 대체하는 것이다. 이 과정에서 set_nice라는 시스템콜을 구현해 process마다 vruntime을 증가하는 양을 조절할 수 있도록 한다. Assignment1에서 수행하였듯 시스템콜을 구현하고, yield(), fork(), kkill()등에서 스케줄링이 필요한 시점에 과제 명세에 따라 vruntime을 nice값을 설정한다.

2. implementation

1. 시스템콜 등록

사용자 영역에서 시스템 콜을 호출할 수 있도록 user/set_nice.c를 구현하고 user/user.h에 함수 원형을 선언하였다. usys.pl에 entry("set_nice"); 추가하여 어셈블리어를 거쳐 커널 모드로 전환하여 systemcall을 호출 할 수 있도록 하였다. Makefile "$U/_set_nice\"을 추가하여 빌드 시 포함되도록 하였다.

아래는 set_nice.c 함수이다.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char **argv)
{
    set_nice(atoi(argv[1]));
    exit(0);
}
```

kernel/syscall.h에 SYS_set_nice에 22라는 시스템콜 번호를 부여하였다.

위 번호를 통해 kernel/syscall.c에서 kernel/sysproc.c에 있는 wrapper함수를 호출 할 수 있다.

Wrapper함수에서 argint()를 사용하여 안전하게 인자값을 받아온 뒤, set_nice(nice);라는 함

수를 호출한다.

```
// add functions
int
set_nice(int nice){
  if (nice < -3 || nice > 2) return 1;
  struct proc *p = myproc();
  p->nice = nice;
  return 0;
}
```

위 함수는 현재 프로세스의 nice값을 인자 값으로 설정하고 0을 반환한다.
단 nice가 과제 명세에 따라 -3보다 작거나 2보다 크다면 1을 반환한다.

2. proc.h에서 proc 구조체에 변수 추가

```
111
112     uint64 vruntime;
113     int nice;
114     struct rb_node run_node;
115   };
116
```

Proc구조체에서 uint64 vruntime을 추가하여 process마다 vruntime의 값을 가지고 있게 하고 int nice로 vruntime의 증감정도를 정할 수 있다. Cfs 스케쥴링에서는 빈번하게 우선순위가 변하기 때문에 red_black tree구조를 활용한다. Linked list를 이용하는 tree구조이기에 struct rb_node run_node 변수를 추가하여 자료구조를 사용할 수 있게 하였다.

rb_node의 주요 변수로는 void* data(현재 process의 포인터), uint64 key(vruntime)이 있다.

3. proc.c에서 helper 함수 구현

```
void
update_vruntime(struct proc *p, uint64 ticks){
  if (p == 0) return;

  if (p->nice > 0){
    p->vruntime += (ticks << p->nice);
  }
  else p->vruntime += (ticks >> (-p->nice));
}
```

스케줄링을 할 때마다 vruntime을 갱신해주어야 하므로 편의를 위해 update_vruntime함수를 만들어주었다. 인자로 process의 주소와 ticks를 받고 그 process의 nice값에 따라 vruntime을 증가시킨다.

- If there are any `RUNNABLE` processes, set the waking process's virtual runtime to the minimum virtual runtime among all `RUNNABLE` processes.
- If there are no `RUNNABLE` processes, but at least one process is currently `RUNNING`, set the waking process's virtual runtime to the lowest virtual runtime of any `RUNNING` process.
- If there are neither `RUNNABLE` nor `RUNNING` processes, set the waking process's virtual runtime to 0.

위 과제 명세에 따르면 runnable process중 vruntime이 가장 작은 process를 고르고, runnable process가 없다면 running에서 vruntime이 가장 작은 process를 고르고, 둘 다 존재하지 않는다면 vruntie을 0으로 설정하여야한다. Vruntime이 가장 작은 vruntime을 반환하는 uint64 get_min_vruntime()함수를 만들어 wakeup()와 kkill() (kkill은 sleeping process를 runnable 상태로 바꾸고 종료를 하여야 하므로) 을 진행할 때마다 호출하여 주었다.

```
uint64
get_min_vruntime(void){
  uint64 min_v = 0xffffffffffffffff;
  int found = 0;

  acquire(&cfs_lock);
  struct rb_node *min_node = rb_first(node: cfs_tree.root);
  if (min_node != 0) {
    min_v = min_node->key;
    found = 1;
  }
  release(&cfs_lock);
  if (found == 1) return min_v;

  struct proc *p;
  for (p = proc; p < &proc[NPROC]; p++){
    if (p->state != RUNNING){
      continue;
    }
    if (min_v > p->vruntime){
      min_v = p->vruntime;
      found = 1;
    }
  }

  if (found == 1) return min_v;
  else return 0;
}
```

4. kernel/proc.c 에서 cfs 스케쥴링 구현

기존 kernel/proc.c에서는 round robin으로 다음 실행할 프로세스를 정하는 스케줄링 방식을 채택하고 있었다. 과제 명세에 따라 이를 vruntime이 가장 작은 프로세스를 선택하여 실행하여야 한다.

```
struct rb_tree cfs_tree;
struct spinlock cfs_lock;
```

여기서 red_black tree에 필요한 변수들을 전역변수로 선언한다.

cfs_lock를 사용하는 이유는 cfs_tree를 critical section으로 설정하여 race condition은 예방하기 위함이다.

```
void
procinit(void)
{
  struct proc *p;
  initlock(&cfs_lock, "cfs_lock");
  initlock(&pid_lock, "nextpid");
  initlock(&wait_lock, "wait_lock");
  for(p = proc; p < &proc[NPROC]; p++) {
      initlock(&p->lock, "proc");
      p->state = UNUSED;
      p->kstack = KSTACK((int) (p - proc));
  }
  cfs_tree.root = 0;
}
```

위는 make qemu를 실행하면 실행되는 기본 함수 중 하나인 procinit()이다.

cfs_lock을 다른 spinlock들과 같은 형식으로 설정해 주고 cfs_tree.root를 0으로 설정하였다.

```
found:
  p->pid = allocpid();
  p->state = USED;
  // modified 처음에 proc에서 vruntime을 0으로 초기화한다.
  p->vruntime = 0;
  p->nice = 0;
```

위는 allocproc()의 수정된 부분이다. 과제 명세에 따라 process의 vruntime과 nice값을 0으로 초기화하였다.

```
// 지금 이 코드가 실행됐다는 건 부모가 있는 경우 이므로 np runtime을 p runtime으로 바꾼다.
np->vruntime = p->vruntime;
np->nice = p->nice;
```

```
//modified
acquire(&cfs_lock);
np->run_node.key = np->vruntime;
np->run_node.data = np;
rb_insert(tree: &cfs_tree, node: &np->run_node);
release(&cfs_lock);
```

위 코드는 kfork()의 추가된 부분이다. 과제 명세에 따라 자식 프로세스가 부모의 vruntime과 nice의 값을 상속받았다. 또한 생성된 프로세스를 cfs_tree에 넣어주기 위해 run_node의 key와 data값을 입력하고 cfs_tree에 insert해주었다.

```
void
yield(void)
{
  struct proc *p = myproc();
  acquire(&p->lock);
  p->state = RUNNABLE;

  //modified
  update_vruntime(p, ticks: 1);
  acquire(&cfs_lock);
  p->run_node.data = p;
  p->run_node.key = p->vruntime;
  rb_insert(tree: &cfs_tree, node: &p->run_node);
  release(&cfs_lock);


  sched();
  release(&p->lock);
}
```

이 코드는 cpu를 양보하는 yield() 함수이다.

Vruntime을 update_vruntime으로 설정해주고, cfs_tree에 넣어준다.

```
void
wakeup(void *chan)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
    if(p != myproc()){
      acquire(&p->lock);
      if(p->state == SLEEPING && p->chan == chan) {
        p->state = RUNNABLE;

        //modified
        p->vruntime = get_min_vruntime();
        acquire(&cfs_lock);
        p->run_node.data = p;
        p->run_node.key = p->vruntime;
        rb_insert(tree: &cfs_tree, node: &p->run_node);
        release(&cfs_lock);
      }
      release(&p->lock);
    }
  }
}
```

```
// to user space (see usertrap() in trap.c).
int
kkill(int pid)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++){
    acquire(&p->lock);
    if(p->pid == pid){
      p->killed = 1;
      if(p->state == SLEEPING){
        // Wake process from sleep().
        p->state = RUNNABLE;

        //modified
        p->vruntime = get_min_vruntime();
        acquire(&cfs_lock);
        p->run_node.data = p;
        p->run_node.key  = p->vruntime;
        rb_insert(tree: &cfs_tree, node: &p->run_node);
        release(&cfs_lock);


      }
      release(&p->lock);
      return 0;
    }
    release(&p->lock);
  }
  return -1;
```

wakeup() 코드는 sleeping상태의 process를 runnable상태로 바꾼다. Runnable한 process는 스케쥴링 대상에 포함되므로 vruntime을 갱신하고 cfs_tree에 넣어주는 역할을 한다.

kkill(int pid)는 프로세스를 종료시킬 지의 여부를 정하는 p->killed를 바꾸는 역할을 한다. 하지만 그 프로세스가 sleeping 상태라면 runnable state로 바꾸고 스케줄링 대상에 포함시켜야하므로, 왼쪽 코드와 마찬가지로 cfs_tree에 insert하는 코드를 추가한다.

```
//modified 기존의 round robin 형태의 for문을 삭제.
// cfs_lock을 전역 락을 걸어서 race condition을 방지.
acquire(&cfs_lock);
struct rb_node *min_node = rb_first(node: cfs_tree.root);
if(min_node != 0){
  p = (struct proc*)min_node->data;
  acquire(&p->lock);

  rb_delete(tree: &cfs_tree, node: &p->run_node);

  release(&cfs_lock); // 트리에서 얻을 정보 다 얻었으니깐 이제 락 해제 접근할 일 이제 X

  if (p->state == RUNNABLE){
    p->state = RUNNING;
    c->proc = p;

    swtch(&c->context, &p->context);
    c->proc = 0;
  }
  release(&p->lock);
} else{
  release(&cfs_lock);
  intr_off();
  asm volatile("wfi");
}
```

위 코드는 핵심인 scheduler() 함수의 수정된 부분이다.

Cfs_tree에서 가장 작은 vruntime을 가진 process를 찾기 위해 cfs_lock을 건다.

만약, min_node가 0이라면 cfs_tree에 있는 process가 없는 것이므로, cfs_lock을 풀고 함수를 종료한다. 여기서 asm volatile(“wfi”);는 진행할 process 가 없으므로 절전모드에 들어가는 것이다.

min_node가 존재한다면 그 rb_node의 구조체의 data에서 process의 주소값을 가져온다.

cfs_tree에서 그 프로세스를 삭제하고 cfs_lock을 풀어준다.

그리고 그 프로세스를 contextswitching 해준다.

3. result

```
$ QEMU: Terminated
inho@jeong-inhoui-MacBookPro project1-vltn617 % make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -sm
0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ cfstest
============================
  CFS Scheduler Test Suite
   (CPUS=1 mode)
============================

=== Test 1: set_nice boundary ===
[PASS] set_nice(-3) returns 0
[PASS] set_nice(-2) returns 0
[PASS] set_nice(-1) returns 0
[PASS] set_nice(0)  returns 0
[PASS] set_nice(1)  returns 0
[PASS] set_nice(2)  returns 0
[PASS] set_nice(-4)  returns 1
[PASS] set_nice(3)   returns 1
[PASS] set_nice(-100) returns 1
[PASS] set_nice(100) returns 1

=== Test 2: nice inheritance via fork ===
[PASS] child inherits parent nice and validates correctly
[PASS] child inherits nice=2, rejects nice=3

=== Test 3: initial nice value ===
[PASS] process starts with valid nice=0

=== Test 4: scheduling order by nice value ===
Expected order: nice=-3 finishes first, nice=2 finishes last
Completion order: -3 0 2
[PASS] lower nice finishes before higher nice

=== Test 5: vruntime inheritance (CPUS=1) ===
Completion order: X then Y
[PASS] lower vruntime child (X) finishes before higher vruntime child (Y)

=== Test 6: wakeup vruntime update ===
woken process completed
[PASS] process wakes from sleep and continues execution

=== Test 7: multi-process stability (CPUS=1) ===
[PASS] 6 processes with different nice values all complete

=== Test 8: nice inheritance chain ===
[PASS] nice inheritance works through fork chain

=== Test 9: kill sleeping process ===
[PASS] kill sleeping process does not crash system

=== Test 10: single process basic operation ===
[PASS] single child process runs and exits cleanly

============================
PASS: 20 / FAIL: 0
```

```
$ usertests cfstest
usertests starting
test cfstest:
[CFS Scheduler Test]

=================================================================
 TEST: 1. Validate return value of set_nice system call
=================================================================
[EXPECT] allows [-3, 2] and returns 1 on fail.

set_nice(-4) -> Result: 1 (Expected: 1) : PASS
set_nice(-3) -> Result: 0 (Expected: 0) : PASS
set_nice(-2) -> Result: 0 (Expected: 0) : PASS
set_nice(-1) -> Result: 0 (Expected: 0) : PASS
set_nice(0) -> Result: 0 (Expected: 0) : PASS
set_nice(1) -> Result: 0 (Expected: 0) : PASS
set_nice(2) -> Result: 0 (Expected: 0) : PASS
set_nice(3) -> Result: 1 (Expected: 1) : PASS

=================================================================
 TEST: 2. Wakeup Test (CPUS=1)
=================================================================
[EXPECT] After child wakes up, P and C should interleave evenly.

PPPPPPPPPPPPPPPPCPPCPCPCPCPCPCPCPCPCPCPCPCC


=================================================================
 TEST: 3. Nice value Fork Inheritance (CPUS=1)
=================================================================
[EXPECT] P and C should interleave evenly (e.g., PPCPCPCP...).

PPCPCCPCPCPCPCPCPCPCPCPCPPCPCPCPCCPPCPCPC


=================================================================
 TEST: 4. Nice value CPU Share (CPUS=1)
=================================================================
[EXPECT] H 16 times, L 1 time, H 16 times ...

LHLHLHLHLHLHLHHLHLLHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHH


=================================================================
 TEST: 5. Check for deadlocks in Multiprocessor environment
=================================================================
[EXPECT] No deadlocks occur & Process ends in A, B, C, D, E, F order or similar

[a: -3] [b: -2] [c: -1] [d: 0] [e: 1] [f: 2]

abcdefcbdcebacbdacdfbeaabcdaccdaeadcbbdccfaabdeebcacdcebbacbacbfceddabcdbebdcdc
cbcdbebaacdcacdeafcbadfbbafacecdabfecddceacabbacfcaddbcebedbecdbfdbaacdbaeccaec
dbacCdeadebabebaddbaffaefbdeabebfddAebedbdfeBdfefdefedefddfeefdedfdefdfedefefdf

================================================
 ALL TESTS COMPLETED.
================================================
PASS
PASS ALL TESTS
```

학우들이 discussion에 올려준 testcase를 돌린 결과이다. 실행 결과, set_nice의 경계값 검증부터 다중 프로세스 환경에서의 스케줄링을 정상적으로 통과했음을 알 수 있다.

4. Troubleshooting

처음에 red_black tree라는 개념이 헷갈려 이 자료구조가 왜 필요한지 어떻게 사용되는지 몰랐다.

cfs 스케줄링에서는 프로세스의 상태변화가 일어날 때마다 해당 process의 vruntime이 빈번하게 갱신된다. 하지만 단순히 linked list구조를 활용할 경우 최솟값을 찾는 탐색에 O(n)의 시간복잡도가 발생하므로 O(log n)의 시간복잡도를 사용하는 red black tree를 활용한다.

해당 자료구조는 삽입할 때 색상 변환과 좌우회전을 통해 정렬되는 자료구조이므로 leftmost한 node를 읽어오면 vruntime이 가장 작음이 보장된다.